

Graph Databases for Digital Twin Systems: Failure Propagation Analysis

**Final Project Report - LOG6953FE
Polytechnique Montréal**

**Anas Dorbani (2424846)
&
Sunny Yasser (2354861)**

Table of Contents

Abstract

Project Deliverables

1. Introduction

1.1 System Context and Purpose

2. Project Overview

3. System Architecture and End-to-End Workflow

3.1 Component Overview

3.2 End-to-End System Workflow

3.3 Digital Twin Characteristics

4. Database Technologies

4.1 Neo4j Implementation Overview

4.2 TachosDB Implementation Overview

4.3 Comparative Analysis Focus

5. Communication Infrastructure

5.1 RabbitMQ Integration

5.2 Message Flow and Routing

6. Visualization Layer

6.1 Godot Implementation

6.2 Building Representation and Failure Visualization

7. Query Processing

7.1 Hop-based Propagation Model

7.2 Query Execution Flow

8. Real-World Applications

8.1 Infrastructure Planning and Risk Assessment

8.2 Disaster Response Simulation

8.3 Utility Network Management

8.4 Urban Development Planning

8.5 Implementation Considerations for Real-World Applications

9. Performance Optimization Focus

10. Future Work

11. Conclusion

List of Figures

Figure 1: System Constellation

Figure 2 - Adapter Service

Figure 3 - Neo4J Internal graph implementation

Figure 4 - Factorization of a cartesian tuple

Figure 5 - RabbitMQ Broker Service

Figure 6 - City Model in Godot

Figure 7 - Affected Buildings

Figure 8 - Runtime TachosDB vs Neo4J

Abstract

This comprehensive report details the implementation of a graph-based system designed to model and visualize failure propagation in interconnected building networks. The project combines graph database technologies with real-time visualization to create a powerful tool for understanding cascading failures in complex systems.

The system compares two distinct graph database implementations: Neo4j, an established graph database using a volcano iterator model, and TachosDB, our novel factorization-based graph database designed specifically for efficient graph traversal operations. The system demonstrates significant runtime improvements through our factorization techniques, enhancing the speed of failure propagation analysis.

The architecture follows a microservices approach, utilizing RabbitMQ for communication between components. A Godot-based front end provides intuitive visualization while backend services handle database operations and performance analysis. This design enables real-time visualization of failure propagation through building networks, with the speed of propagation visually representing the actual computational performance differences between the database implementations.

This project represents an important advancement in physical modeling technology for infrastructure analysis, offering practical applications for failure prediction and theoretical contributions to graph database optimization.

Project Deliverables

The repository for the project is available [here](#). The steps required to deploy are available in the README file. The link for the video presentation of the project has been added [here](#).

1. Introduction

Physical modeling of interconnected systems has emerged as a powerful approach for analyzing complex infrastructure networks in virtual environments. In critical infrastructure settings such as urban environments, understanding the propagation of failures across interconnected components is crucial for both preventative maintenance and disaster response planning.

This project implements a sophisticated physical modeling system that combines advanced graph database technologies with real-time visualization to model failure propagation in building networks. By utilizing graph structures to represent buildings and their connections, the system can simulate how failures cascade through networks, providing valuable insights for infrastructure planning and emergency response.

A key innovation of this project is the comparative analysis of different graph database technologies. We contrast the industry-standard Neo4j graph database with our novel TachosDB implementation, which utilizes factorization techniques to achieve improved runtime performance for specific graph queries. This comparison not only demonstrates the feasibility of specialized graph databases for infrastructure modeling applications but also provides quantitative performance metrics that guide visualization speeds.

System Context and Purpose

While our system shares some characteristics with digital twin technology, it's important to clarify that our implementation is specifically a physical model rather than a digital twin or digital shadow:

- **Physical Model:** Our system creates a virtual representation of a connected system without direct connection to real-world assets. It models a fictional city and simulates theoretical failure propagation based on the modeled connections.
- **Simulation Focus:** The system focuses on simulating theoretical failure scenarios rather than replicating real-world data or assets.
- **Comparative Analysis Tool:** Beyond infrastructure analysis, the system serves as a platform for comparing database performance in graph-traversal operations.

This clarification is important as it positions the system correctly within the spectrum of virtual modeling approaches and sets appropriate expectations for its applications.

2. Project Overview

The primary goal of this project is to create a physical modeling system that can:

- Model complex building networks using graph structures
- Simulate failure propagation through these networks
- Visualize the propagation in real-time
- Compare runtime performance characteristics of different graph database implementations

The system allows users to select one or more buildings as the starting points for failure propagation. It then calculates the spread of failure through connected buildings using graph traversal algorithms implemented in two separate database systems: Neo4j and TachosDB. The results are visualized in a Godot-based 3D environment, where the speed of visualization corresponds to the actual computational performance of the underlying database systems.

This approach not only provides a practical tool for failure analysis but also serves as a visual demonstration of the runtime improvements achieved through our factorization-based database implementation. The project combines multiple technologies, including graph databases,

message queuing systems, 3D visualization, and custom algorithms, into a cohesive modeling platform.

3. System Architecture and End-to-End Workflow

The system follows a microservices architecture designed to enable a comprehensive end-to-end workflow from user interaction to visualization of results. Here, we detail the complete process flow and component interactions.

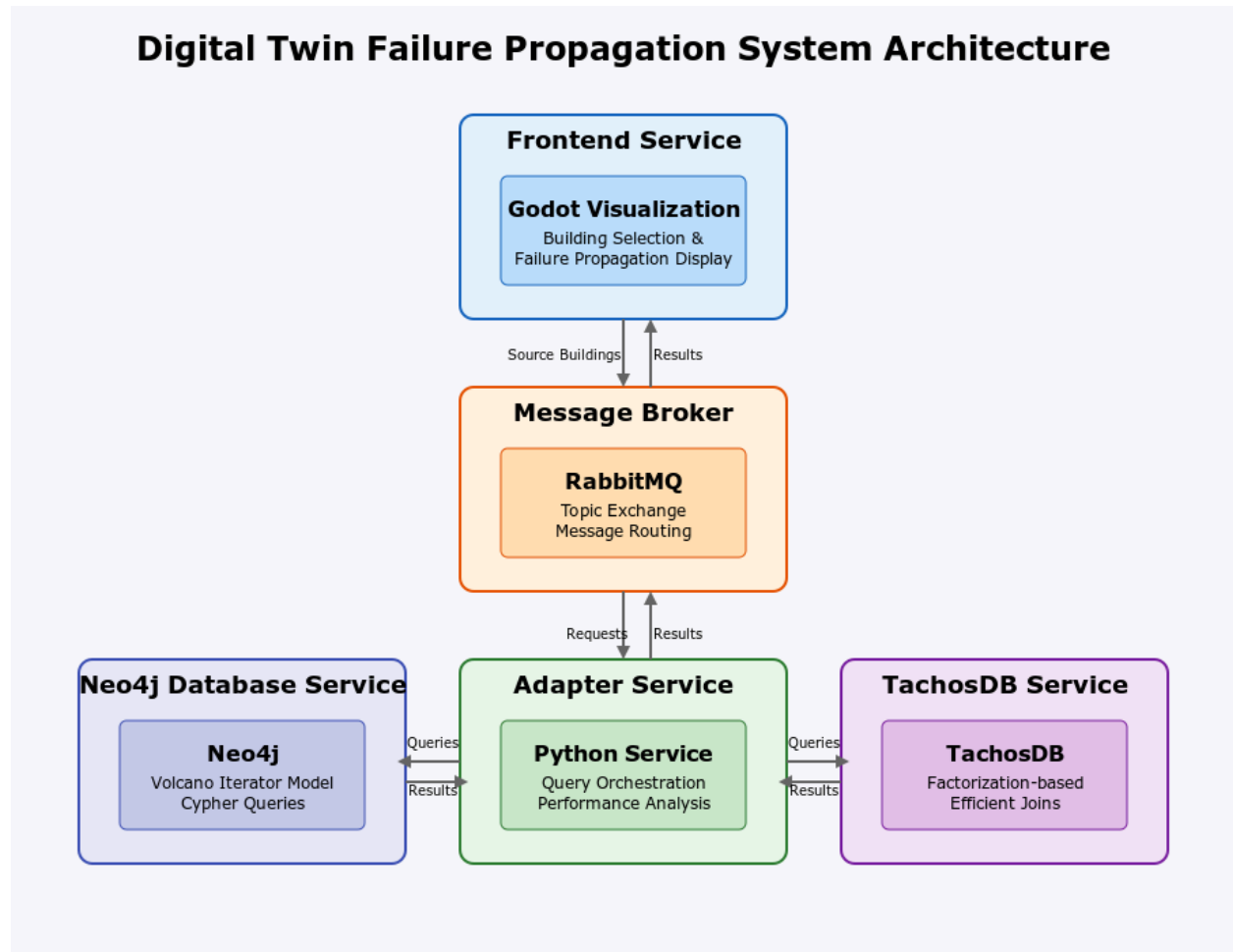


Figure 1 - System Constellation

3.1 Component Overview

The system consists of the following key components:

- **Godot Visualization Front-End:** A 3D visualization environment that displays buildings and their connections, allows user selection of failure sources, and visualizes the propagation of failures.
- **RabbitMQ Message Broker:** Facilitates communication between front-end and back-end services, handling message routing and delivery.
- **Adapter Service:** Coordinates between the visualization layer and database services, processing requests and results, and calculating performance metrics.

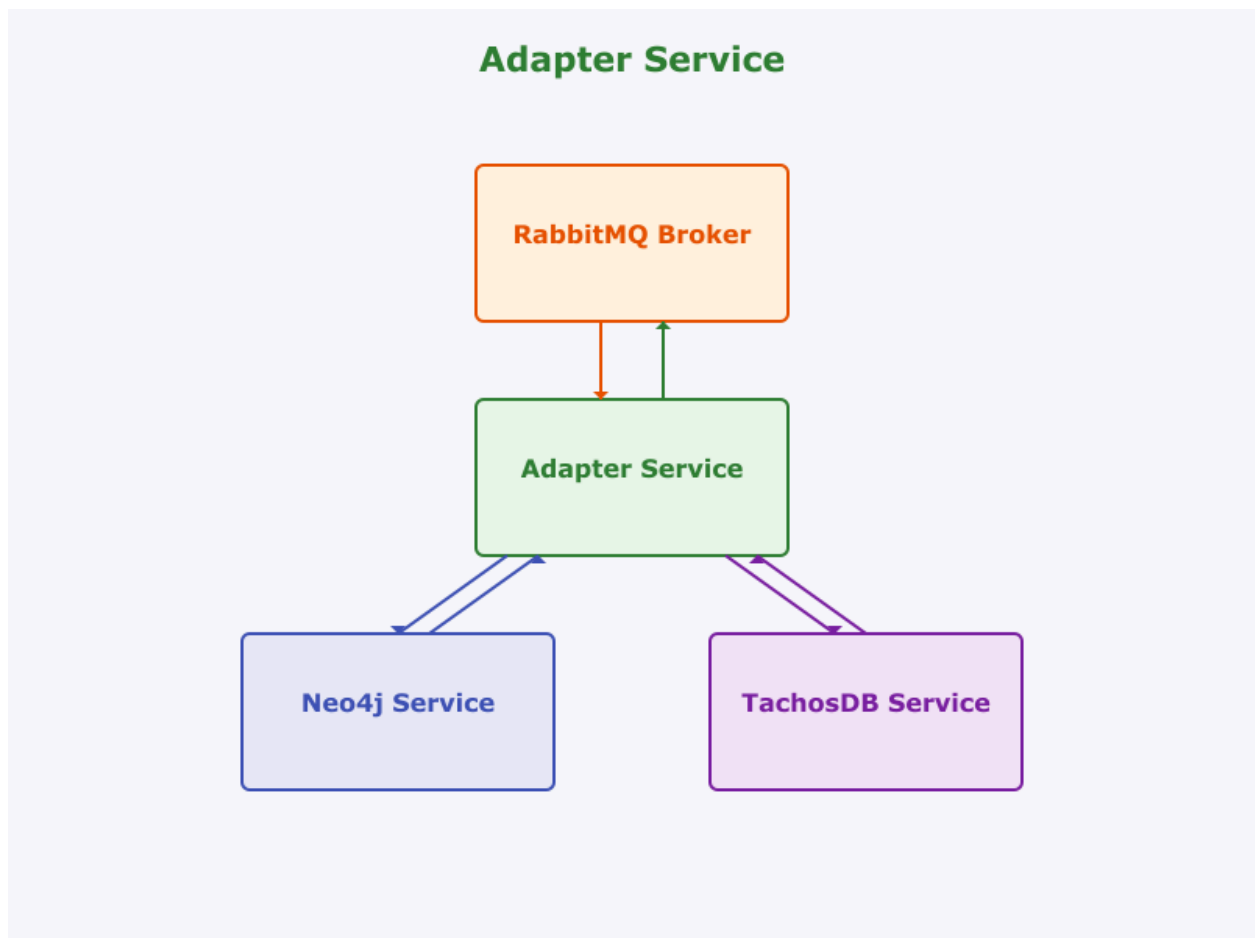


Figure 2 - Adapter Service

- **Database Services:**
 - **Neo4j Service:** Executes graph queries using the Neo4j graph database
 - **TachosDB Service:** Executes identical queries using our custom factorization-based graph database

3.2 End-to-End System Workflow

The complete workflow of the system proceeds as follows:

1. System Initialization:

- Both database services load the graph model of the fictional city network
- The visualization service constructs a 3D representation of buildings and connections
- The adapter service establishes connections to RabbitMQ and both database services

2. User Interaction and Failure Selection:

- Users interact with the 3D visualization to inspect the city infrastructure
- Buildings are represented as 3D structures with visible connections between them
- Users select one or more buildings as initial failure points by clicking on them
- Selected buildings are highlighted in the visualization

3. Query Initiation and Routing:

- The visualization front-end captures selected building IDs
- These IDs are formatted as a JSON message and published to RabbitMQ
- The message includes a timestamp and routing information
- RabbitMQ routes the message to the adapter service

4. Parallel Query Processing:

- The adapter service receives the building IDs
- It formats identical query requests for both database services
- Queries are sent simultaneously to both Neo4j and TachosDB services
- Each database service executes the same set of hop-based propagation queries
- Both services track execution times for performance comparison

5. Results Collection and Analysis:

- Each database service returns:
 - Lists of affected buildings at each hop level
 - Execution time statistics for each query
- The adapter service collects these results
- It calculates performance ratios between the two database implementations
- Results and performance metrics are formatted as structured messages

6. Result Dissemination:

- The adapter publishes results to RabbitMQ
- Separate messages contain:
 - Affected building lists from Neo4j
 - Affected building lists from TachosDB
 - Performance comparison metrics
- RabbitMQ routes these messages to the visualization service

7. Dynamic Visualization:

- The visualization service receives the results
- It maps database results to visual elements in the 3D environment
- Failure propagation is animated through the network
- The speed of propagation visualization is adjusted based on performance ratios
- Fast database performance is represented by faster visual propagation

8. User Analysis and Iteration:

- Users observe the propagation patterns and speeds
- They can select different starting points to test various failure scenarios
- The system maintains a history of scenarios for comparison
- Users can export results for further analysis

This end-to-end workflow creates a seamless experience from user interaction to visualization while maintaining a clear separation between components through the message-based architecture.

3.3 Digital Twin Characteristics

Characteristic Code	Characteristic Name	Description
MC1	System Under Study	Simulates a fictional city with interconnected buildings modeled in a graph.
MC2	Physical Acting Components	Simulated actions like propagation of failure through building connections.
MC6	Digital Twin Services	Offers failure analysis, real-time visualization, and performance benchmarking.
MC7	Twinning Time-scale	Visualization speed reflects actual query execution time (Neo4j vs TachosDB).
MC10	Digital Twin Models and Data	Graph-based data models in Neo4j and TachosDB represent buildings and links.
MC11	Tooling and Enablers	Uses Neo4j, TachosDB, RabbitMQ, Godot, and a custom adapter service.
MC12	Digital Twin Constellation	Microservices architecture with clear component interaction and messaging.
MC17	Insights and Decision Making	Supports scenario testing and comparative performance analysis.

4. Database Technologies

4.1 Neo4j Implementation Overview

Neo4j is an industry-standard graph database that utilizes a property graph model. In our system, Neo4j serves as the baseline implementation for failure propagation queries, using the Cypher query language to express multi-hop traversals.

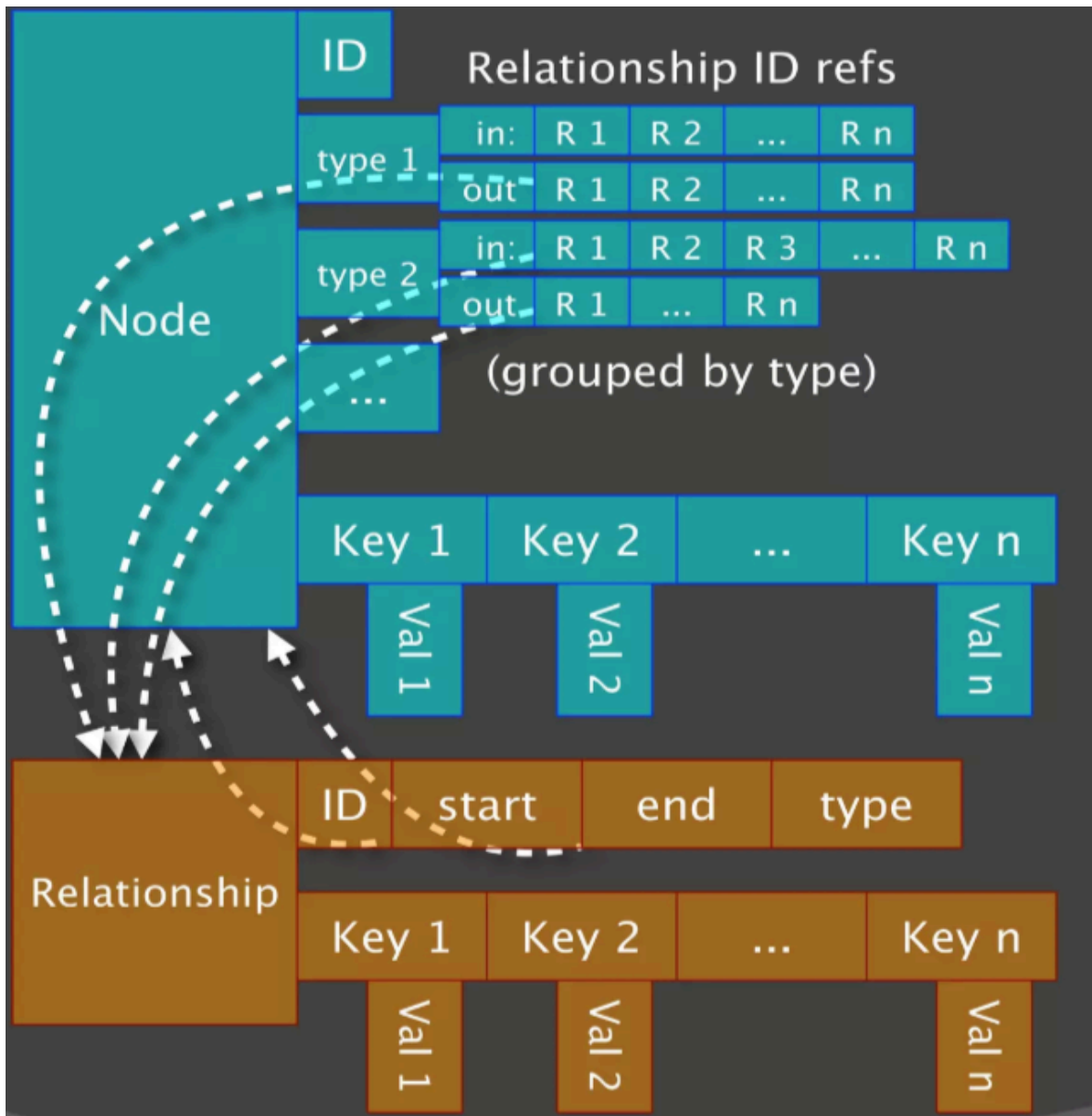


Figure 3 - Neo4J Internal graph implementation

4.2 TachosDB Implementation Overview

TachosDB is our custom graph database implementation designed specifically for optimized graph traversal operations. It employs factorization techniques and optimized join algorithms specifically designed to improve runtime performance for the types of queries common in failure propagation analysis.

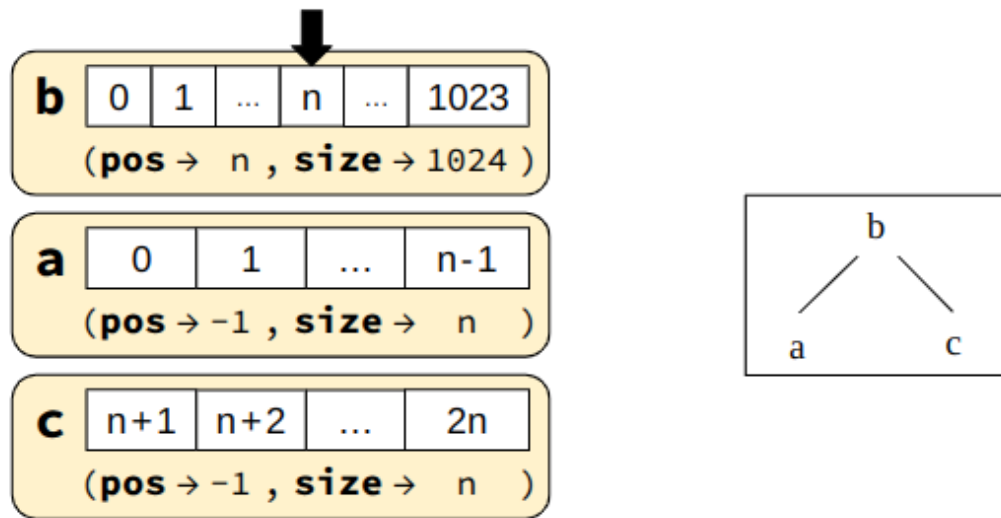


Figure 4 - Factorization of a cartesian tuple

4.3 Comparative Analysis Focus

Our comparative analysis focuses specifically on runtime performance between the two database implementations:

- **Execution Speed:** For multi-hop queries on densely connected graphs, TachosDB outperforms Neo4j by optimizing query execution paths.
- **Scaling Properties:** As the number of hops increases, the runtime performance gap between TachosDB and Neo4j widens.

The performance differences are visually represented in the system, with the speed of failure propagation in the visualization corresponding to the actual query execution speed of each database system.

5. Communication Infrastructure

5.1 RabbitMQ Integration

RabbitMQ serves as the central message broker in our system, facilitating communication between the Godot front-end and the back-end services. This messaging approach enables loose coupling between components and provides reliability and flexibility in message routing.

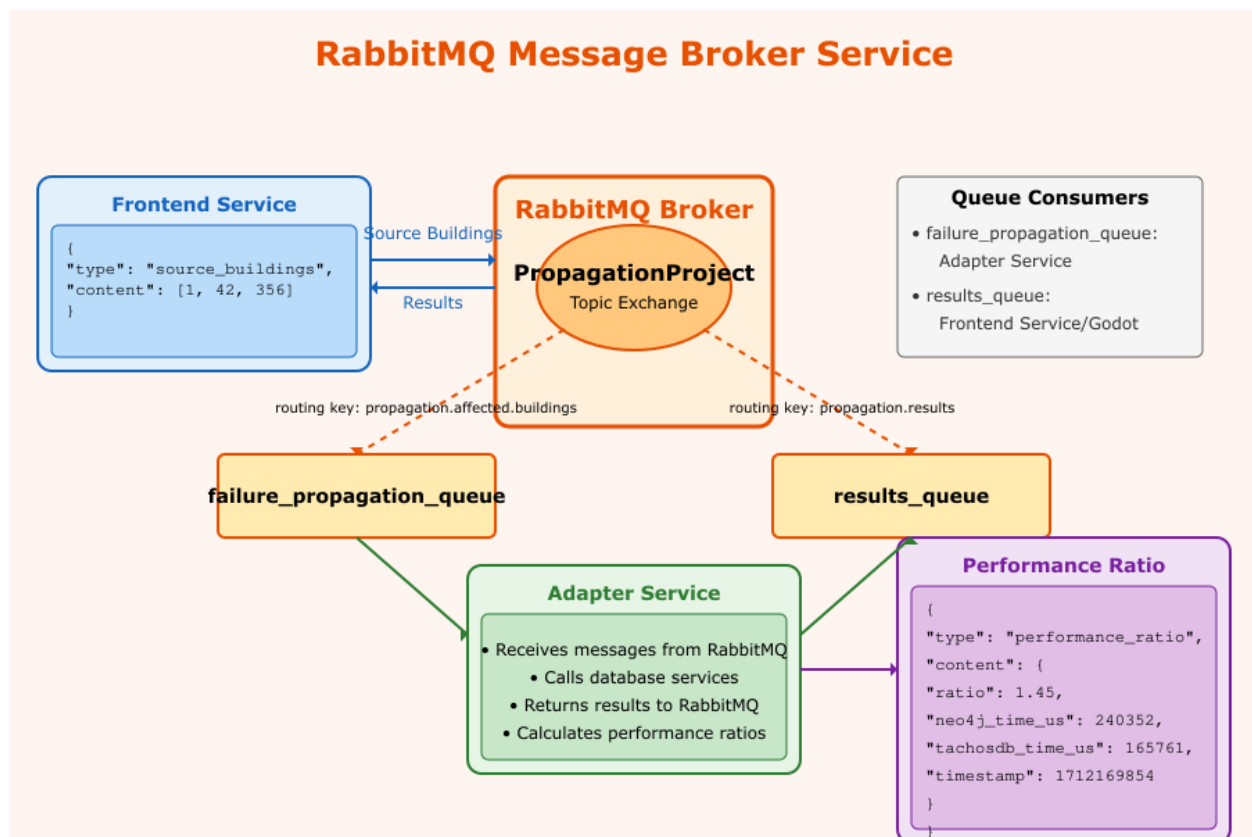


Figure 5 - RabbitMQ Broker Service

5.2 Message Flow and Routing

The system employs a sophisticated message routing strategy with specific message types and routing patterns:

- **Building Selection Messages:** Carry user-selected failure source buildings from the frontend to the backend
- **Database Result Messages:** Return affected buildings and performance statistics for each database implementation
- **Performance Ratio Messages:** Provide comparative metrics between the database implementations

This messaging infrastructure ensures efficient communication while maintaining component independence, allowing for scalability and flexibility in the system architecture.

6. Visualization Layer

6.1 Godot Implementation

The visualization component creates an intuitive 3D representation of the building network, allowing users to:

- View the complete building network in 3D
- Select buildings as failure sources
- Observe the propagation of failures through the network
- Compare performance between database implementations

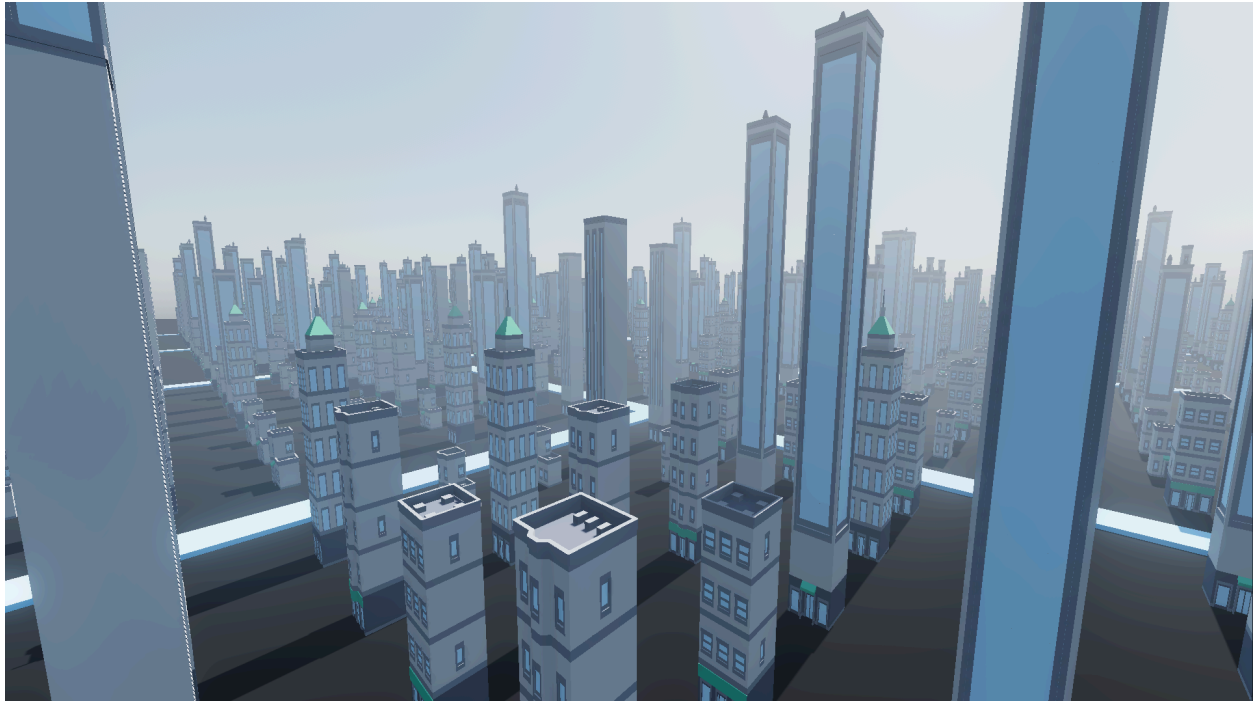


Figure 6 - City Model in Godot

6.2 Building Representation and Failure Visualization

The visualization maps database entities to visual elements and creates an intuitive representation of failure propagation:

- Buildings are represented as 3D models with status indicators
- Connections between buildings are visualized as paths or lines
- Failure propagation is shown through color changes and visual effects
- Propagation timing reflects actual database performance differences

The propagation of failure is visualized using a timed sequence of visual effects:

1. Source buildings (selected by the user) are highlighted immediately
2. Affected buildings are highlighted in waves corresponding to hop levels
3. The speed of these waves reflects the query execution performance

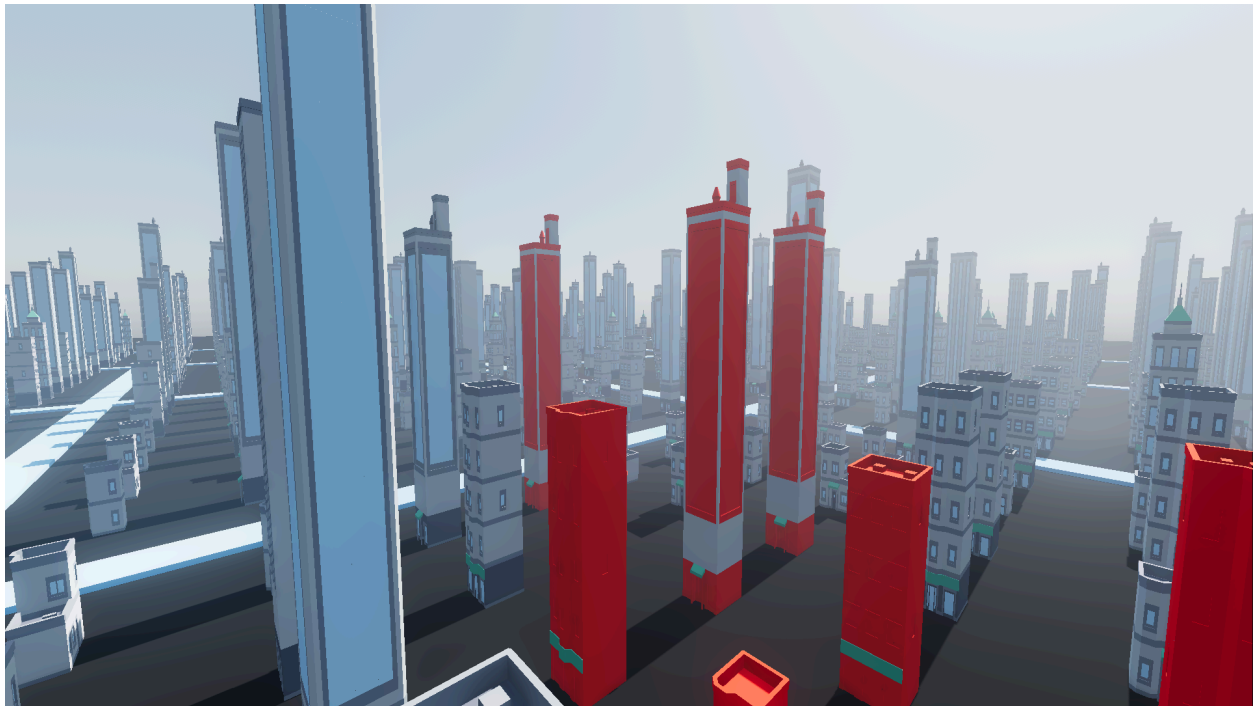


Figure 7 - Affected Buildings

7. Query Processing

7.1 Hop-based Propagation Model

Our failure propagation model uses a hop-based approach, where a "hop" represents a direct connection between buildings. The system executes queries for 1-hop through 4-hop propagation, simulating how failures spread through the network at increasing distances from the source.

```
// Level 1 - One-hop neighbors
```

```

MATCH (n:Building)
WHERE n.id IN $input_values
MATCH (n)-[:CONNECTED_TO]->(neighbor:Building)
WHERE NOT neighbor.id IN $input_values // exclude input nodes
RETURN collect(DISTINCT neighbor.id) AS nodes

// Level 2 - Two-hop neighbors
MATCH (n:Building)
WHERE n.id IN $input_values
MATCH (n)-[:CONNECTED_TO]->()-[:CONNECTED_TO]->(neighbor:Building)
RETURN collect(DISTINCT neighbor.id) AS nodes

// Level 3 - Three-hop neighbors
MATCH (n:Building)
WHERE n.id IN $input_values
MATCH (n)-[:CONNECTED_TO]->()-[:CONNECTED_TO]->()-[:CONNECTED_TO]->(neighbor:Building)
RETURN collect(DISTINCT neighbor.id) AS nodes

// Level 4 - Four-hop neighbors
MATCH (n:Building)
WHERE n.id IN $input_values
MATCH (n)-[:CONNECTED_TO]->()-[:CONNECTED_TO]->()-[:CONNECTED_TO]->()-[:CONNECTED_TO]->
(neighbor:Building)
RETURN collect(DISTINCT neighbor.id) AS nodes

```

7.2 Query Execution Flow

When a user selects failure source buildings, the system executes increasingly complex graph traversal queries to determine affected buildings at each hop level. These queries are executed on both database systems simultaneously, allowing for direct performance comparison.

8. Real-World Applications

8.1 Infrastructure Planning and Risk Assessment

While our system models a fictional city, the approach has direct applications for accurate infrastructure planning:

- **Critical Node Identification:** The system can help identify which infrastructure nodes, if compromised, would cause the most widespread cascading failures. This information is valuable for prioritizing protection measures and redundancy planning.
- **Resilience Testing:** Urban planners can use similar systems to test the resilience of proposed infrastructure designs by simulating failures at various points and analyzing the extent of propagation.
- **Evacuation Planning:** Understanding how failures propagate through urban infrastructure can inform emergency evacuation planning, helping authorities determine which areas might become isolated or compromised during disasters.

8.2 Disaster Response Simulation

The system provides a foundation for disaster response training and planning:

- **Response Prioritization:** Emergency services can use similar systems to simulate infrastructure failures and determine which affected areas should receive priority attention.
- **Resource Allocation:** By understanding the pattern and speed of failure propagation, authorities can better position emergency resources in advance of disasters.
- **Training Scenarios:** The visual nature of the system makes it suitable for creating training scenarios for emergency management personnel, helping them understand complex infrastructure interdependencies.

8.3 Utility Network Management

The approach demonstrated in our system can be adapted for specific utility networks:

- **Power Grid Analysis:** Energy companies can model how failures in electrical infrastructure might cascade through the system, informing maintenance prioritization and upgrade planning.
- **Water System Vulnerability:** Municipal water authorities can analyze how contamination or pressure loss might propagate through water distribution networks.
- **Telecommunications Resilience:** Network operators can identify critical nodes in the communication infrastructure where failures would have the most widespread impact.

8.4 Urban Development Planning

For urban development scenarios, our approach offers valuable insights:

- **Growth Impact Assessment:** Urban planners can analyze how new developments might affect the resilience of existing infrastructure networks.
- **Connectivity Optimization:** Transportation planners can optimize new connections to minimize the potential for widespread failure propagation while maximizing accessibility.
- **Phased Development Planning:** The system can help planners understand how infrastructure resilience changes through different phases of urban development.

8.5 Implementation Considerations for Real-World Applications

To adapt our approach for real-world applications, several considerations are important:

- **Data Acquisition:** Real implementations require accurate mapping of infrastructure connections, which involves integration with GIS systems, CAD models, and existing infrastructure databases.
- **Calibration:** The propagation models would need to be calibrated based on historical failure data and engineering specifications of actual infrastructure components.
- **Dynamic Updates:** Real-world systems would benefit from the ability to update the infrastructure model as changes occur, requiring integration with asset management systems.
- **Multi-layer Integration:** Real infrastructure involves multiple interconnected layers (power, water, transportation, etc.), which would require more complex modeling of cross-domain dependencies.

These real-world applications demonstrate the potential value of our approach beyond the current implementation, showing how the core technology could be adapted to address critical infrastructure challenges in various domains.

9. Performance Optimization

A key innovation in our system is the focus on runtime optimization for graph traversal operations. We measure execution time for queries at different hop levels and calculate performance ratios between Neo4j and TachosDB.

This focus on runtime performance is critical for practical applications of failure propagation analysis, where rapid assessment of potential impacts is essential for response planning. The visualization system makes these performance differences intuitive by directly mapping query execution speed to the visual propagation of failures.

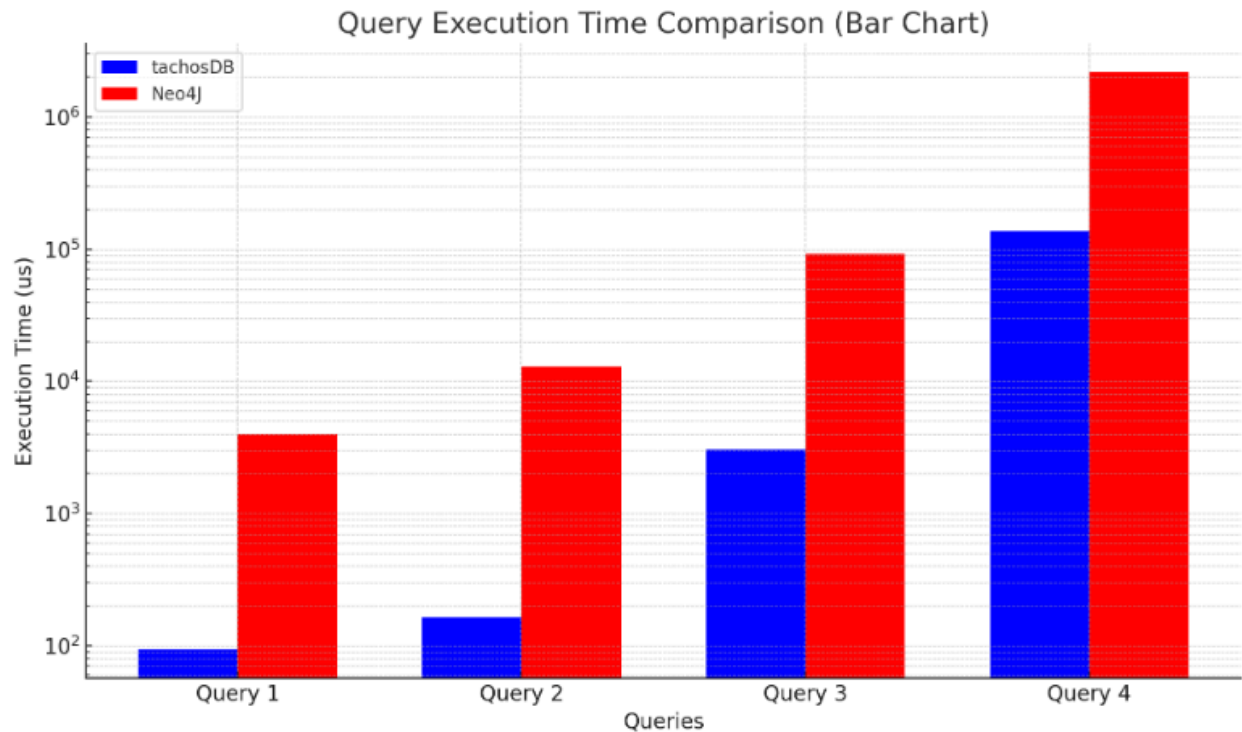


Figure 8 - Runtime TachosDB vs Neo4J

10. Future Work

Several promising directions for future research and development have been identified:

1. **Integration with Real Infrastructure Data:** Adapting the system to work with real urban infrastructure data from GIS systems would enhance its practical applicability.
2. **Predictive Modeling:** Incorporating machine learning to predict potential failure points based on network topology and historical data.
3. **Multi-domain Infrastructure Modeling:** Extending the system to model interactions between different infrastructure types (power, water, transportation, etc.).
4. **Interactive Scenario Testing:** Enhancing the user interface to support complex "what-if" scenario development and testing.

5. **Performance Optimization:** Further refinement of query execution strategies to support even larger infrastructure networks.
6. **Mobile and Web Interfaces:** Developing additional interfaces to make the system accessible to emergency planners in the field.

These future directions would enhance the system's capabilities and broaden its applicability to real-world infrastructure planning and management.

11. Conclusion

This project demonstrates the value of specialized graph database technologies for infrastructure modeling, particularly in analyzing failure propagation through interconnected systems. By combining established graph database capabilities with our novel factorization approach, we have created a system that efficiently models failure propagation and visually represents the performance advantages of our optimized implementation.

The microservices architecture provides flexibility and scalability, while the intuitive visualization interface makes complex infrastructure relationships and failure patterns accessible to users without specialized database knowledge.

Key contributions of this project include:

- A comprehensive physical modeling system for analyzing failure propagation in building networks
- A novel factorization-based graph database implementation optimized for improved runtime performance
- A direct comparison of different graph database approaches in a practical application context
- A visualization system that intuitively represents both failure propagation and performance differences

This work provides a foundation for future research and development in graph-based infrastructure modeling, with potential applications in urban planning, emergency response, and resilience engineering. By focusing on runtime optimization and intuitive visualization, the project demonstrates how advanced database technologies can enhance our understanding of complex infrastructure systems and their vulnerabilities.